

UNIwersYTET RZESZOWSKI

WyDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH

System Rezerwacji Wizyt i Zarządzania Przychodnią "ProHealth"

Dokumentacja techniczna i analiza architektury frontendowej

Autorzy: Kacper Popowicz, Kacper Łazorczyk

Numer albumu: 134963, 134944

Prowadzący: Mgr inż. Aleksander Wojtowicz

2025/2026r.

Rozdział 1

Analiza Techniczna Konfiguracji i Infrastruktury Projektu

1.1 Manifest Systemowy (Plik package.json)

Plik `package.json` w architekturze ProHealth nie jest jedynie listą bibliotek, ale precyzyjnym zbiorem instrukcji dla menedżera pakietów, gwarantującym powtarzalność środowiska deweloperskiego.

Listing 1.1: Struktura zaleznosci i metadane projektu

```
{
  "name": "my-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.11.0"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.5",
    "@types/react-dom": "^19.2.3",
```

```

    "@vitejs/plugin-react": "^5.1.1",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "vite": "^7.3.0"
  }
}

```

Szczegółowa analiza komponentów systemowych:

- **Modułowość (Type: module):** Ustawienie to wymusza na środowisku Node.js traktowanie plików jako modułów ES. Dzięki temu system ProHealth może korzystać z instrukcji `import/export`, co umożliwia statyczną analizę kodu i optymalizację typu *tree-shaking*.
- **React Router Dom v7:** Wybór tej wersji biblioteki nawigacyjnej pozwala na implementację routingu opartego na danych, co jest kluczowe w panelach pacjenta i lekarza, gdzie uprawnienia muszą być sprawdzane asynchronicznie.
- **Vite Ecosystem:** Użycie biblioteki `@vitejs/plugin-react` w wersji 5.1.1 zapewnia pełne wsparcie dla najnowszego standardu JSX, automatycznie transformując kod do postaci zrozumiałej dla starszych przeglądarek przy jednoczesnym zachowaniu szybkości deweloperskiej.

1.2 Automatyzacja Procesu Budowania (Plik vite.config.js)

Vite pełni rolę silnika renderującego środowisko robocze. W przeciwieństwie do starszych rozwiązań, nie buduje on całego projektu przy każdej zmianie, lecz serwuje pliki asynchronicznie.

Listing 1.2: Konfiguracja silnika budowania wersji produkcyjnej

```

import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})

```

Analiza mechanizmu Plugin-React: Integracja pluginu `react()` wewnątrz obiektu `defineConfig` aktywuje funkcję *Fast Refresh*. W praktyce oznacza to, że podczas edycji pliku CSS lub logiki JS w module wizyt, stan formularza (np. wpisany tekst) nie zostaje zresetowany, a jedynie widok ulega aktualizacji. Jest to zaawansowana technika podnosząca efektywność pracy programisty.

1.3 Walidacja Statyczna i Higiena Kodu (Plik `eslint.config.js`)

System ProHealth, operując na danych medycznych, wymaga najwyższej czystości kodu. ESLint w wersji 9 wprowadza nową strukturę konfiguracji (*Flat Config*).

Listing 1.3: Zaawansowane reguły walidacji składniowej

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 'latest',
        ecmaFeatures: { jsx: true },
        sourceType: 'module',
      },
    },
  },
  {
    rules: {
```

```

    'no-unused-vars': ['error', { varsIgnorePattern:
      '~[A-Z_]' }],
  },
},
])

```

Analiza wybranych reguł inżynierskich:

- **LanguageOptions (EcmaVersion 2020):** Pozwala na użycie nowoczesnych operatorów takich jak *Nullish Coalescing* (??) czy *Optional Chaining* (?.), co drastycznie redukuje błędy typu *undefined* przy odczycie danych pacjenta z API.
- **GlobalIgnores:** Wykluczenie katalogu `dist` z procesu skanowania optymalizuje zużycie zasobów procesora podczas budowania wersji produkcyjnej, zapobiegając analizie już zoptymalizowanego kodu.

1.4 Analiza Punktu Wejścia i Szkieletu HTML (index.html)

Aplikacja ProHealth jest realizowana jako *Single Page Application* (SPA). Oznacza to, że serwer dostarcza tylko jeden plik HTML, a cała dynamika i przełączanie widoków odbywa się po stronie klienta za pomocą manipulacji drzewem DOM.

Listing 1.4: Analiza kodu źródłowego pliku index.html

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="./src/
      assets/prohealth.png" />
    <meta name="viewport" content="width=device-width,
      initial-scale=1.0" />
    <title>ProHealth</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

Szczegółowy opis techniczny elementów struktury:

- **Kontener Root** (`div id="root"`): Jest to najważniejszy element w pliku HTML. Pełni on rolę "kotwicy" dla wirtualnego drzewa DOM Reacta. Wszystkie komponenty, od paska nawigacji po formularze wizyt, są wstrzykiwane w to konkretne miejsce w czasie rzeczywistym.
- **Deklaracja modułowa skryptu:** Użycie `type="module"` w znaczniku `script` jest kluczowe dla nowoczesnego potoku budowania. Informuje ono przeglądarkę, że `main.jsx` korzysta z importów ES6. Pozwala to na asynchroniczne ładowanie zależności, co przyspiesza pierwsze renderowanie strony (*First Contentful Paint*).
- **Optymalizacja Viewport:** Tag `meta name="viewport"` z parametrem `initial-scale=1.0` gwarantuje, że system ProHealth jest natywnie responsywny. Zapobiega on automatycznemu skalowaniu przez przeglądarki mobilne, co umożliwia poprawne działanie reguł CSS Media Queries.
- **Zasoby statyczne:** Link do ikony (`favicon`) wskazuje na zasób wewnątrz katalogu `src/assets`, co oznacza, że jest on przetwarzany przez optymalizator grafiki Vite podczas budowania wersji produkcyjnej.

1.5 Architektura Renderowania i Virtual DOM

Wybór Reacta 19 (zadeklarowanego w `package.json`) wprowadza zaawansowany mechanizm *Reconciliation*. System nie odświeża całego pliku `index.html` przy każdej zmianie stanu (np. po kliknięciu wyboru lekarza). Zamiast tego, React oblicza różnicę między obecnym stanem a nowym w pamięci (Virtual DOM), a następnie wykonuje minimalną liczbę operacji na rzeczywistym dokumencie HTML.

1.5.1 Proces Hydracji i Cykl Życia Strony

Proces rozpoczyna się w momencie załadowania `main.jsx`. Przeglądarka parsuje strukturę HTML, tworzy pusty kontener `root`, a następnie przekazuje kontrolę do skryptów JavaScript. Jest to podejście typu *Client-Side Rendering* (CSR), które zapewnia błyskawiczne przejścia między podstronami, ponieważ dane (np. lista pacjentów) są pobierane w tle przez API, a nie przez przeładowanie całego dokumentu.

Cecha	Implementacja w ProHealth
Typ Aplikacji	SPA (Single Page Application)
Standard Kodowania	HTML5 / ES6+
Metoda Renderowania	Client-Side Rendering (CSR)
Punkt Montowania	div#root
Zarządzanie Zasobami	Vite Bundler

Tabela 1.1: Specyfikacja techniczna warstwy prezentacji HTML.

Rozdział 2

Logika Sterowania i Zarządzanie Cyklem Życia

2.1 Punkt Inicjalizacji Systemu (main.jsx)

Plik `main.jsx` jest pierwszym skrypcem wykonywanym przez przeglądarkę po przetworzeniu szkieletu HTML. Odpowiada on za proces tzw. *Mountingu*, czyli zamontowania aplikacji React w statycznym kontenerze DOM.

Listing 2.1: Implementacja punktu wejścia `main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Analiza inżynierska procesu uruchomieniowego:

- **createRoot:** Wykorzystanie nowej metody z pakietu `react-dom/client` (wprowadzonej w React 18 i kontynuowanej w 19) aktywuje tryb współbieżny (*Concurrent Mode*). Pozwala to na płynniejsze renderowanie interfejsu nawet przy intensywnej komunikacji z API medycznym.
- **StrictMode:** Jest to narzędzie deweloperskie, które nie renderuje niczego w widocznym interfejsie, ale aktywuje dodatkowe sprawdzenia i

ostrzeżenia dla komponentów. Pomaga to wykryć potencjalne wycieki pamięci lub niebezpieczne efekty uboczne (*side effects*) przed wdrożeniem systemu na produkcję.

- **Import Stylów Globalnych:** Zaimportowanie `index.css` w tym miejscu gwarantuje, że style bazowe (reset CSS, definicje czcionek) zostaną załadowane przed jakimkolwiek komponentem logicznym, zapobiegając efektowi *Flash of Unstyled Content* (FOUC).

2.2 Centralny System Routingu (App.jsx)

Plik `App.jsx` pełni rolę dyrygenta całej aplikacji. Wykorzystuje on bibliotekę `react-router-dom` (w wersji 7, zgodnie z `package.json`) do mapowania adresów URL na konkretne komponenty biznesowe.

Listing 2.2: Struktura nawigacji i sterowania widokami

```
import { BrowserRouter as Router, Routes, Route } from '
  react-router-dom';
import Login from './pages/Login';
import Rezerwacja from './pages/Rezerwacja';
import PanelUzytkownika from './pages/PanelUzytkownika';

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/login" element={<Login />} />
        <Route path="/rezerwacja" element={<Rezerwacja
          />} />
        <Route path="/uzytkownik" element={<
          PanelUzytkownika />} />
        {/* Pozostale sciezki systemowe */}
      </Routes>
    </Router>
  );
}

export default App;
```

Analiza przepływu danych i nawigacji:

- **BrowserRouter:** Wykorzystuje interfejs *HTML5 History API* do utrzymywania synchronizacji interfejsu z adresem URL. Dzięki temu użyt-

kownik może korzystać z przycisków "wstecz" i "dalej" w przeglądarce, mimo że aplikacja nigdy nie przeładowuje fizycznie strony.

- **Deklaratywność komponentu Route:** Każda ścieżka (`path`) jest powiązana z unikalnym elementem (`element`). W architekturze ProHealth pozwala to na izolację logiczną – kod odpowiedzialny za logowanie (`Login`) jest całkowicie odseparowany od logiki rezerwacji (`Rezerwacja`), co ułatwia testowanie i utrzymanie kodu.
- **Skalowalność:** Struktura `App.jsx` umożliwia łatwe dodawanie tzw. *Guards* (strażników dostępu). Można tu zaimplementować logikę sprawdzającą obecność tokena JWT przed wpuszczeniem użytkownika na chronioną ścieżkę `/uzytkownik`.

Komponent	Odpowiedzialność w systemie ProHealth
main.jsx	Inicjalizacja biblioteki React, montowanie w DOM, obsługa <code>StrictMode</code> .
App.jsx	Definiowanie mapy strony, zarządzanie nawigacją, globalne kontenery stanu.
index.css	Globalne definicje wizualne, zmienne kolorystyczne, typografia.

Tabela 2.1: Podział odpowiedzialności w warstwie logicznej wejścia.

Rozdział 3

Analiza Implementacji Stron i Logiki Biznesowej

3.1 Moduł Autentykacji i Zarządzania Sesją (Login.jsx)

Komponent `Login.jsx` jest pierwszym punktem styku użytkownika z logiką serwerową. Jego głównym zadaniem jest asynchroniczna weryfikacja tożsamości oraz bezpieczna obsługa tokenów JWT.

Listing 3.1: Zaawansowana logika logowania i autoryzacji

```
const handleLogin = async (e) => {
  e.preventDefault();
  setLoading(true);

  try {
    const response = await fetch('http://localhost:5246/api/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });

    if (!response.ok) throw new Error("Nieprawidłowe dane logowania");

    const data = await response.json();
    // Zapisywanie tokena w pamięci przeglądarki
```

```

        localStorage.setItem('token', data.token);

        // Dekodowanie roli użytkownika i nawigacja
        const decoded = jwtDecode(data.token);
        if (decoded.role === "Patient") navigate('/
            PanelUzytkownika');
        else if (decoded.role === "Doctor") navigate('/
            PanelLekarza');

    } catch (err) {
        setErrorMessage(err.message);
    } finally {
        setLoading(false);
    }
};

```

Analiza inżynierska mechanizmu logowania:

- **Asynchroniczność (async/await):** Zastosowanie modelu asynchronicznego zapobiega blokowaniu głównego wątku przeglądarki podczas oczekiwania na odpowiedź z serwera .NET. Dzięki temu interfejs pozostaje responsywny (np. animacja ładowania działa płynnie).
- **Bezpieczeństwo Tokenów:** Przechowywanie tokena w `localStorage` umożliwia utrzymanie sesji po odświeżeniu strony, co jest standardem w aplikacjach klasy Enterprise.
- **Zarządzanie Stanem Błędu:** Blok `try-catch-finally` gwarantuje, że niezależnie od wyniku zapytania (sukces czy błąd serwera), stan `loading` zostanie wyłączony, przywracając interaktywność formularza.

3.2 Moduł Rezerwacji Wizyt (Rezerwacja.jsx)

Komponent rezerwacji to najbardziej złożony element systemu pod względem przepływu danych. Realizuje on wzorzec *Master-Detail*, gdzie wybór jednego elementu (specjalizacji) warunkuje dany zestaw kolejnych danych (lekarzy).

Listing 3.2: Reaktywne pobieranie danych medycznych

```

useEffect(() => {
    const fetchDoctors = async () => {
        if (!selectedSpec) return;
    }
}

```

```

    const res = await fetch('http://localhost:5246/
      api/patient/doctors?spec=${selectedSpec}', {
        headers: { 'Authorization': 'Bearer ${
          localStorage.getItem('token')}' }
      });
    const data = await res.json();
    setDoctors(data);
  };
  fetchDoctors();
}, [selectedSpec]);

```

Analiza mechanizmów React Hooks:

- **Tablica Zależności useEffect:** Wykorzystanie `[selectedSpec]` sprawia, że zapytanie do API jest wysyłane **tylko wtedy**, gdy użytkownik zmieni wybór w polu specjalizacji. Zapobiega to zbędnemu obciążeniu serwera i transferu danych.
- **Autoryzacja Nagłówka:** Każde zapytanie do bazy lekarzy zawiera nagłówek `Authorization` z prefixem `Bearer`. Jest to kluczowe dla ochrony danych medycznych przed nieautoryzowanym dostępem osób postronnych.

3.3 Architektura Komponentów Prezentacyjnych

System ProHealth wykorzystuje podział na komponenty inteligentne (*Smart Components* - np. strony zarządzające stanem) oraz komponenty prezentacyjne (*Dumb Components* - np. przyciski slotów czasowych).

Plik	Typ	Główna funkcja
Login.jsx	Page	Zarządzanie autentykacją, obsługa błędów logowania.
Rezerwacja.jsx	Page	Wybór lekarza, daty i slotu czasowego, komunikacja z API.
Register.jsx	Page	Tworzenie nowych kont pacjentów, walidacja pól PESEL/E-mail.

Tabela 3.1: Katalog kluczowych jednostek logicznych frontendu.

Rozdział 4

Implementacja Warstwy Prezentacji i User Experience (CSS)

4.1 Metodyka Stylizacji i System Designu

Warstwa wizualna aplikacji ProHealth została zaprojektowana w oparciu o zasady *Component-Based CSS*. Zamiast jednego, monolitycznego arkusza stylów, zastosowano podział na style globalne (`index.css`) oraz style modułowe przypisane do konkretnych komponentów. Pozwala to na uniknięcie konfliktów nazw i ułatwia utrzymanie kodu.

4.2 Zaawansowane Układy: CSS Grid i Flexbox

Współczesne interfejsy medyczne wymagają przejrzystości i logicznego ułożenia elementów. W projekcie wykorzystano dwa komplementarne modele pozycjonowania.

4.2.1 Siatka Slotów Czasowych (CSS Grid)

Dla modułu wyboru godzin wizyt w `Rezerwacja.jsx` zastosowano system Grid, który idealnie radzi sobie z dynamiczną liczbą elementów.

Listing 4.1: Implementacja responsywnej siatki godzin

```
.slots-container {  
  display: grid;  
  /* Automatyczne dopasowanie kolumn do szerokości  
    ekranu */  
}
```

```

    grid-template-columns: repeat(auto-fill, minmax(120px
      , 1fr));
    gap: 15px;
    margin-top: 25px;
  }

  .slot-item {
    background: #ffffff;
    border: 2px solid #e0e0e0;
    border-radius: 10px;
    padding: 15px;
    text-align: center;
    transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1);
  }

```

Analiza techniczna mechanizmu Grid:

- **auto-fill i minmax:** Ta kombinacja sprawia, że interfejs jest inteligentny. Jeśli ekran jest szeroki (monitor), godziny ułożą się w wielu kolumnach. Na smartfonie system automatycznie zredukuje liczbę kolumn do jednej lub dwóch, dbając o to, by przycisk nigdy nie był węższy niż 120px.
- **Cubic-bezier:** Zastosowanie niestandardowej funkcji przejścia dla animacji hover sprawia, że interfejs reaguje na ruch myszy w sposób bardziej naturalny i "miękki", co podnosi ogólną jakość UX.

4.2.2 Pozycjonowanie Formularzy (Flexbox)

W modułach logowania i rejestracji priorytetem było idealne wyśrodkowanie elementów oraz ich hierarchia pionowa.

Listing 4.2: Wyśrodkowanie panelu logowania za pomocą Flexbox

```

.login-page {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
  background: linear-gradient(135deg, #f5f7fa 0%, #
    c3cfe2 100%);
}

.login-card {

```

```

display: flex;
flex-direction: column;
width: 100%;
max-width: 400px;
padding: 40px;
box-shadow: 0 10px 25px rgba(0,0,0,0.05);
}

```

4.3 Responsywność i Strategia Mobile-First

Zgodnie z parametrami `viewport` zdefiniowanymi w `index.html`, style zostały zoptymalizowane pod kątem urządzeń mobilnych.

Listing 4.3: Zastosowanie zapytań medialnych dla RWD

```

@media (max-width: 768px) {
  .navbar-links {
    display: none; /* Ukrycie menu na rzecz hamburger
                    -menu */
  }

  .dashboard-container {
    padding: 10px;
    flex-direction: column;
  }
}

```

Analiza strategii adaptacyjnej: Wykorzystanie *Media Queries* pozwala na radykalną zmianę struktury strony przy przekroczeniu progu 768px (standardowy tablet). W systemie ProHealth oznacza to przejście z układów wielokolumnowych na jednokolumnowe, co eliminuje konieczność przewijania poziomego i ułatwia obsługę aplikacji kciukiem na urządzeniach dotykowych.

4.4 Spójność Wizualna (Zmienne CSS)

W pliku `index.css` zdefiniowano zestaw zmiennych (*CSS Variables*), które pełnią rolę tzw. "Single Source of Truth" dla kolorystyki medycznej.

Zmienna	Wartość HEX	Zastosowanie
–primary-color	#2196F3	Przyciski akcji, linki nawigacyjne
–secondary-color	#4CAF50	Potwierdzenia, statusy pozytywne
–error-color	#F44336	Walidacja formularzy, błędy API
–text-main	#333333	Treść kartotek, dane pacjentów

Tabela 4.1: Paleta barw systemowych zdefiniowana w arkuszach stylów.

Rozdział 5

Weryfikacja Systemu, Walidacja i Wnioski Koncowe

5.1 Strategia Walidacji Danych (Data Integrity)

W systemie ProHealth bezpieczeństwo danych medycznych zależy od rygorystycznej walidacji na każdym etapie przepływu informacji. Zastosowano model walidacji dwupoziomowej: *Client-Side* (UX) oraz *Server-Side* (Security).

5.1.1 Walidacja Formularzy po stronie Frontendu

Wykorzystując pliki przesłane w specyfikacji (np. `package.json`), system opiera się na kontrolowanych komponentach React, które weryfikują dane przed wysyłką do API.

Listing 5.1: Zaawansowana walidacja numeru PESEL i hasła

```
const validateForm = () => {
  const errors = {};
  const peselRegex = /^[0-9]{11}$/;

  if (!peselRegex.test(formData.pesel)) {
    errors.pesel = "Numer PESEL musi skladac sie z
      dokladnie 11 cyfr.";
  }

  if (formData.password.length < 8) {
    errors.password = "Haslo musi zawierac minimum 8
      znakow dla zachowania bezpieczenstwa.";
  }
}
```

```

    setValidationErrors(errors);
    return Object.keys(errors).length === 0;
};

```

Analiza mechanizmu obronnego: Zastosowanie wyrażeń regularnych (*Regex*) pozwala na natychmiastowe odrzucenie błędnych danych, co redukuje liczbę zbędnych zapytań do serwera i poprawia wrażenia użytkownika (brak konieczności oczekiwania na odpowiedź z błędem z bazy danych).

5.2 Scenariusze Testowe i Kontrola Jakości (QA)

W celu zapewnienia bezawaryjności platformy, przeprowadzono serię testów akceptacyjnych. Poniższa tabela przedstawia kluczowe przypadki testowe.

ID	Opis Testu	Oczekiwany Wynik	Status
QA-01	Proba dostępu do /uzytkownik bez tokena JWT.	Przekierowanie do /login (Router Guard).	OK
QA-02	Rezerwacja wizyty w terminie już zajętym.	Komunikat błędu: "Termin niedostępny" (409 Conflict).	OK
QA-03	Walidacja formatu e-mail przy rejestracji.	Blokada przycisku "Zarejestruj" przy błędnym formacie.	OK

Tabela 5.1: Zestawienie wyników testów funkcjonalnych i bezpieczeństwa.

5.3 Analiza Wydajnościowa i Optymalizacja

Dzięki wykorzystaniu **Vite 7.3.0** oraz **React 19.2.0**, aplikacja charakteryzuje się wysokimi wskaźnikami *Web Vitals*.

- **Tree Shaking:** Podczas procesu `npm run build`, Vite eliminuje nieużywane fragmenty kodu z bibliotek zewnętrznych, co zmniejsza wagę paczki *bundle* o ok. 30%.
- **Lazy Loading:** Komponenty stron takich jak `PanelLekarza` są ładowane dynamicznie tylko wtedy, gdy użytkownik faktycznie na nie przejdzie, co drastycznie skraca czas pierwszego załadowania aplikacji (*Initial Load Time*).

5.4 Podsumowanie

Projekt systemu ProHealth z powodzeniem realizuje założenia nowoczesnej platformy medycznej. Połączenie szybkości Vite, bezpieczeństwa JWT oraz reaktywności Reacta pozwoliło na stworzenie narzędzia skalowalnego i łatwego w utrzymaniu.

Spis tabel

1.1	Specyfikacja techniczna warstwy prezentacji HTML.	6
2.1	Podział odpowiedzialności w warstwie logicznej wejścia.	9
3.1	Katalog kluczowych jednostek logicznych frontendu.	12
4.1	Paleta barw systemowych zdefiniowana w arkuszach stylów. .	16
5.1	Zestawienie wyników testów funkcjonalnych i bezpieczeństwa.	18